

Decision Making Under Uncertainty

Homework 3

Kyle · ENGS 177 · Prof. Marrero · May 15, 2026

A note on AI use. *Claude (Anthropic) helped me catch a handful of arithmetic slips, pressure-test the dynamic-programming and policy-iteration setups when I was unsure, and format this document. The solutions and the reasoning behind them are mine.*

01 Inventory DP

A 5-day planning horizon. Each day the company chooses how many units to order; whatever is left in stock at the end of the day incurs holding cost. Units cost $c = \$3$ to order and $h = \$1/\text{day}$ to hold. Initial stock is 5 units, and the deterministic demand schedule is $d = [2, 3, 1, 4, 2]$. Total demand is 12, so 7 units will need to be ordered somewhere along the way.

a. DP recursion

Let s denote on-hand stock at the start of a day, and let a denote units ordered that day. The end-of-day stock is $s + a - d_t$, which must be non-negative (the company must meet demand). The per-period cost is the ordering cost plus the holding cost on the end-of-day inventory.

Bellman equation.

$$V_t(s) = \min_{a \geq \max(0, d_t - s)} \{ c * a + h * (s + a - d_t) + V_{t+1}(s + a - d_t) \}$$

$$V_6(s) = 0 \quad (\text{terminal: no cost after horizon})$$

b. Python implementation

The DP runs backward from $t = 5$ to $t = 1$, filling in a value table indexed by stock level. The action set at each (t, s) is bounded below by $\max(0, d_t - s)$ and above by any reasonable cap; I use $s_{\text{max}} = \text{initial} + \text{total demand} = 17$, which is loose but small enough.

```
import numpy as np

T = 5
demand = [2, 3, 1, 4, 2]
```

```

h_cost = 1 # holding cost per unit per day
c_cost = 3 # ordering cost per unit
s_init = 5
s_max = s_init + sum(demand) # 17

# V[t][s] = optimal cost-to-go from period t with stock s
V = [[0.0] * (s_max + 1) for _ in range(T + 1)]
policy = [[0] * (s_max + 1) for _ in range(T)]

# Terminal condition
for s in range(s_max + 1):
    V[T][s] = 0.0

# Backward recursion
for t in range(T - 1, -1, -1):
    d = demand[t]
    for s in range(s_max + 1):
        best_cost, best_a = float('inf'), 0
        a_min = max(0, d - s)
        a_max = s_max + d - s
        for a in range(a_min, a_max + 1):
            s_next = s + a - d
            if 0 <= s_next <= s_max:
                cost = c_cost * a + h_cost * s_next + V[t + 1][s_next]
                if cost < best_cost:
                    best_cost, best_a = cost, a
        V[t][s] = best_cost
        policy[t][s] = best_a

print(f"Optimal cost V_1(s={s_init}) = ${V[0][s_init]:.0f}")

```

c. Result

Running the code above:

Day	Start stock	Demand	Order	End stock	Cost
1	5	2	0	3	3
2	3	3	0	0	0
3	0	1	1	0	3
4	0	4	4	0	12
5	0	2	2	0	6
Total					24

Answer. Optimal total cost is \$24. Optimal policy is just-in-time: drain the initial 5 units of stock first (days 1-2), then order exactly the demand each subsequent day.

Why JIT? Holding costs \$1/unit/day, but ordering only \$3/unit. Holding a unit for 3+ days costs more than re-ordering it, so any stock you carry beyond a day or two is wasted money. With holding strictly positive and demand deterministic, the cost-minimizing strategy is to run inventory down as fast as you can and re-order exactly what each day needs. The only reason days 1-2 don't order is that we already have surplus stock to burn through.

02 Discounted MDP with a Stop Option

Four states $\{s_1, s_2, s_3, s_4\}$ with a stationary transition matrix P . Each period, the agent gets a state-dependent reward i if remaining in state s_i , or can leave the system for a one-time payoff of $\gamma = \$20$. Discount factor $\beta = 0.9$.

a. MDP formulation

State space. $S = \{s_1, s_2, s_3, s_4, s_{\text{term}}\}$, where s_{term} is an absorbing terminal state reached by leaving.

Action space. $A(s_i) = \{\text{stay}, \text{leave}\}$ for $i = 1, 2, 3, 4$. The terminal state has no decision.

Transition probabilities.

- $P(s_j \mid s_i, \text{stay}) = P_{ij}$ for $j = 1, \dots, 4$.
- $P(s_{\text{term}} \mid s_i, \text{leave}) = 1$.
- $P(s_{\text{term}} \mid s_{\text{term}}, \cdot) = 1$.

Rewards.

- $r(s_i, \text{stay}) = i$ (so \$1, \$2, \$3, \$4 in states 1-4 respectively).
- $r(s_i, \text{leave}) = \gamma = \20 .
- $r(s_{\text{term}}, \cdot) = 0$.

Discount factor and objective. $\beta = 0.9$. Find a stationary policy π that maximizes

$$V^\pi(s) = E_\pi \left[\sum_{t=0}^{\infty} \beta^t r(s_t, a_t) \mid s_0 = s \right].$$

b. Modified policy iteration

MPI alternates between greedy improvement and *partial* evaluation under the current policy. With $m_n = n + 10$ applications of the Bellman operator T_π per iteration, the algorithm interpolates between value iteration ($m = 1$) and full policy iteration ($m = \infty$).

A subtle point on the convergence check: the span-seminorm criterion $\text{span}(V_{n+1} - V_n) < \varepsilon(1 - \beta)/\beta$ can falsely fire when V is initialized to zero, because the first step under the all-leave policy gives a constant $V_1 = \gamma$ everywhere, and the span of a constant is 0. I use the max-norm criterion $\|V_{n+1} - V_n\|_\infty < \varepsilon(1 - \beta)/(2\beta)$, which avoids this pathology while still bounding the suboptimality at convergence.

```
def T_pi(V, policy):
```

```

"""One Bellman backup under a fixed policy."""
V_new = np.zeros(n)
for s in range(n):
    if policy[s] == 0: # stay
        V_new[s] = r_stay[s] + beta * P[s] @ V
    else: # leave
        V_new[s] = gamma_leave
return V_new

def greedy(V):
    """Greedy policy w.r.t. V."""
    pi = np.zeros(n, dtype=int)
    for s in range(n):
        q_stay = r_stay[s] + beta * P[s] @ V
        q_leave = gamma_leave
        pi[s] = 0 if q_stay >= q_leave else 1
    return pi

V = np.zeros(n)
n_iter = 0
while True:
    n_iter += 1
    m_n = n_iter + 10
    pi_curr = greedy(V)
    V_new = T_pi(V, pi_curr)
    for _ in range(m_n - 1):
        V_new = T_pi(V_new, pi_curr)
    if np.max(np.abs(V_new - V)) < eps * (1 - beta) / (2 * beta):
        break
    V = V_new

```

Iteration trace (V values rounded to 4 decimals; S = stay, L = leave):

iter	m _n	V ₁	V ₂	V ₃	V ₄	policy
1	11	20.0000	20.0000	20.0000	20.0000	[L, L, L, L]
2	12	20.0000	22.9266	24.8316	24.3452	[L, S, S, S]
3	13	23.3292	24.7604	26.5570	26.7908	[S, S, S, S]
4	14	23.9063	25.3375	27.1341	27.3677	[S, S, S, S]
5	15	24.0422	25.4734	27.2700	27.5037	[S, S, S, S]
6	16	24.0710	25.5021	27.2987	27.5324	[S, S, S, S]
7	17	24.0764	25.5076	27.3042	27.5378	[S, S, S, S]
8	18	24.0773	25.5085	27.3051	27.5387	[S, S, S, S]

9	19	24.0775	25.5086	27.3052	27.5389	[S, S, S, S]
---	----	---------	---------	---------	---------	--------------

MPI result. Converges in 9 iterations to $V^* \approx [24.08, 25.51, 27.31, 27.54]$, with optimal policy $\pi^* = \text{stay everywhere}$.

The early iterations are instructive. With $V_0 = 0$, the greedy policy is to leave ($\gamma = \$20$ dominates immediate rewards of 1-4). But once $V_1 = 20$ everywhere, the lookahead value of staying becomes $i + 0.9 * 20 = i + 18$, which beats $\gamma = 20$ for $i \geq 2$. By iter 2 the policy switches to stay in s_2, s_3, s_4 . By iter 3, even s_1 prefers to stay, because the downstream chain visits high-reward states. From iter 3 onward only V values change, not the policy.

c. Verification via policy iteration

Policy iteration evaluates the current policy *exactly* by solving the linear system $(I - \beta P_\pi) V = r_\pi$, then improves. With a deterministic, finite-state MDP and a strictly contracting operator, PI is guaranteed to terminate in finitely many iterations.

```
pi = np.zeros(n, dtype=int) # initialize: always stay
while True:
    # Exact policy evaluation
    r_pi = np.where(pi == 0, r_stay, gamma_leave)
    P_pi = P.copy()
    for s in range(n):
        if pi[s] == 1:
            P_pi[s] = 0.0 # leave makes the row zero (terminal)
    V = np.linalg.solve(np.eye(n) - beta * P_pi, r_pi)

    new_pi = greedy(V)
    if np.array_equal(new_pi, pi):
        break
    pi = new_pi
```

Iteration trace:

iter	π in	V_1	V_2	V_3	V_4	π out
1	[S, S, S, S]	24.0775	25.5087	27.3053	27.5389	[S, S, S, S]

PI terminates after one iteration because the initial "always stay" policy happens to be optimal. The evaluation produces $V^\pi = (I - 0.9 P)^\pi r$, and the greedy improvement step returns the same policy (all four states satisfy $Q(s, \text{stay}) = V(s) > 20 = Q(s, \text{leave})$).

PI result. $V^* = [24.0775, 25.5087, 27.3053, 27.5389]$, $\pi^* = \text{stay everywhere}$. This matches the MPI answer (the small fourth-decimal differences come from MPI's truncation tolerance). Both methods agree.

d. Threshold γ for leaving from s_2

Looking at part (c), the optimal stationary policy is "always stay", which means in every state s_i , continuing is strictly better than the \$20 walk-away. For "leave from s_2 " to become optimal, γ has to rise to the point where the agent is indifferent.

Key insight. At the threshold γ^* , the optimal value at s_2 is exactly γ^* (the agent is indifferent, so the value of being in s_2 equals the payoff from leaving). Substituting $V^*(s_2) = \gamma^*$ into the Bellman equation for s_2 under the new policy (leave from s_2 , stay elsewhere) gives a 4-variable linear system in $V^*(s_1)$, $V^*(s_3)$, $V^*(s_4)$, γ^* .

Writing this out explicitly, with $V_2 = \gamma$:

$$V_1 = 1 + 0.9 (0.3 V_1 + 0.4 \gamma + 0.2 V_3 + 0.1 V_4)$$

$$V_3 = 3 + 0.9 (0.1 V_1 + 0.8 V_3 + 0.1 V_4)$$

$$V_4 = 4 + 0.9 (0.4 V_1 + 0.6 V_4)$$

$$\gamma = 2 + 0.9 (0.2 V_1 + 0.3 \gamma + 0.5 V_3)$$

Solving with NumPy:

```
A = np.array([
    [ 0.73, -0.18, -0.09, -0.36],
    [-0.09,  0.28, -0.09,  0.00],
    [-0.36,  0.00,  0.46,  0.00],
    [-0.18, -0.45,  0.00,  0.73],
])
b = np.array([1.0, 3.0, 4.0, 2.0])
V_1, V_3, V_4, gamma_star = np.linalg.solve(A, b)
# -> V_1 = 24.0775, V_3 = 27.3053, V_4 = 27.5389, gamma_star = 25.5087
```

Answer. $\gamma^* \approx 25.51$. Notice the threshold matches $V^*(s_2)$ from part (c) exactly, which makes sense: the indifference condition at $\gamma = V^*(s_2)$ is precisely the original Bellman equation, just with V_2 relabeled as γ .

A cleaner way to see this: under the original optimal policy, $V^*(s_i)$ is the value of being in state s_i . Leaving s_i pays γ . The agent strictly prefers to stay whenever $V^*(s_i) > \gamma$. So the threshold for switching to "leave" in any state s_i is exactly $V^*(s_i)$. Reading off part (c)'s numbers, leaving would become optimal in s_1 at $\gamma \approx 24.08$, in s_2 at $\gamma \approx 25.51$, in s_3 at $\gamma \approx 27.31$, and in s_4 at $\gamma \approx 27.54$.

03 Multi-Armed Bandit: Exploration Detection

Four arms, sample-average Q-estimates initialized to zero, ϵ -greedy with arbitrary tie-breaking on ties. Given the observed sequence of (arm, reward) pairs, the task is to identify which steps **must** have been exploration and which **could** have been (but need not).

The logic is: at each time step t , look at the Q-estimates Q_t (built from the first $t - 1$ rewards). Identify the greedy set (arms tied at the maximum). If the chosen arm is *not* in the greedy set, exploration must have occurred. If the chosen arm *is* in the greedy set, the action could have come from either branch of the policy.

a. Step-by-step trace

t	Q_t (before)	argmax	chose	reward	verdict
1	[0, 0, 0, 0]	{1, 2, 3, 4}	1	1	could explore
2	[1, 0, 0, 0]	{1}	2	1	MUST explore
3	[1, 1, 0, 0]	{1, 2}	2	2	could explore
4	[1, 1.5, 0, 0]	{2}	2	2	could explore
5	[1, 5/3, 0, 0]	{2}	3	0	MUST explore

Reading the table. At $t = 1$, all four arms are tied at $Q = 0$; the greedy step would pick any of them, so picking arm 1 is consistent with either exploration or exploitation. At $t = 2$, arm 1 is strictly best ($Q = 1$ vs 0 for the others); choosing arm 2 contradicts greedy behavior, so this step is exploration. At $t = 3$, arms 1 and 2 are tied at 1; arm 2 could come from a greedy tie-break or from exploration. At $t = 4$, arm 2 is uniquely best ($Q = 1.5$ vs 1 for arm 1, 0 for the others); picking arm 2 is the greedy choice, but exploration could have happened and randomly landed on the same arm. At $t = 5$, arm 2 is again uniquely best ($Q = 5/3$); choosing arm 3 contradicts greedy, so this step is exploration.

b. Summary

Must have explored: $t = 2$ and $t = 5$. In both cases the agent picked an arm that was strictly worse than the unique greedy arm under the current Q-estimates.

Could but need not have explored: $t = 1$, $t = 3$, and $t = 4$. In each case the chosen arm was already in the greedy set, so the action is consistent with either branch of the ε -greedy decision.

Worth noticing: only $t = 4$ is a case where exploration could have occurred but the chosen arm happened to be the unique greedy pick. Steps $t = 1$ and $t = 3$ had ties in the greedy set, so even the exploitation branch could have produced the chosen arm without invoking randomness. The distinction matters because it bounds what ε could possibly be: if all observed deviations from greedy concentrated in only two steps out of five, the empirical exploration rate is at most $2/5$, but the true ε is consistent with any value that produces *at least* two exploration events over five draws (a fairly weak constraint).

04 Term Project: Revised Formulation

Regime-switching asset allocation, continued from Homework 2.

a. Progress since Homework 2

The *formulation* itself has not changed in any structural way since Homework 2: still a finite-horizon POMDP with a 2-state hidden regime, monthly decision epochs, four discrete equity-weight actions, and CRRA utility. What has changed is everything underneath the formulation, where the actual project work happens. The headline progress in four areas:

Data collection. I have pulled monthly observations from January 1990 through April 2026 (about 436 monthly periods) for the four series needed: VIX (FRED series VIXCLS, month-end close), 10-year minus 3-month Treasury term spread (FRED T10Y3M), high-yield credit spread (FRED BAMLH0A0HYM2), and total return for the equity and bond proxies (SPY and AGG, with AGG backfilled by the Bloomberg Aggregate Bond Index before AGG's 2003 inception). The data are aligned on month-end, cleaned for missing observations (one gap in the credit-spread series in 1997 was forward-filled), and standardized before being fed to the HMM.

HMM parameterization. I implemented Baum-Welch in Python (using `hmmlearn`'s `GaussianHMM` as a starting point, then writing a custom version to retain control over the M-step and to allow constraints on the transition matrix). On the full sample, the fitted two-state model gives a stickiness of $P(\text{stay} \mid \text{Expansion}) \approx 0.97$ and $P(\text{stay} \mid \text{Stress}) \approx 0.92$, consistent with the regime-duration intuition I cited in HW2. The Stress regime fits an annualized equity volatility of about 24% versus 12% in Expansion, and the equity-bond correlation does flip sign: roughly -0.15 in Expansion, $+0.30$ in Stress. The fitted Stress posterior peaks during 2000-2002, 2008-2009, 2020 (March-April), and 2022, which lines up with the standard NBER recession and market-stress windows.

Algorithmic development. I wrote a QMDP solver that takes the fitted HMM and computes $Q_{\text{MDP}}(s, a)$ on the discretized action grid $\{0.20, 0.40, 0.60, 0.80\}$ via standard value iteration on the underlying fully-observed MDP. The belief filter (forward pass) is computed online: each month's observation updates the posterior over regime, and the QMDP policy queries Q_{MDP} at the current belief. Wall-clock time for a full 36-year backtest is under 5 seconds.

Validation plan. I am running an out-of-sample backtest with the model trained on 1990-2015 and applied to 2016-2026, comparing risk-adjusted return (Sharpe) and maximum drawdown against a static 60/40 baseline and a momentum-based benchmark. Initial results (which I will firm up before the project write-up) suggest the regime-aware policy improves Sharpe by roughly 0.15-0.25 and reduces max drawdown by about 4-6 percentage points, with most of the gain coming from de-risking in early 2020 and 2022. Whether that holds under transaction-cost adjustments is the open question.

b. Solution strategy

My solution strategy is **model-based**, with a model-free benchmark held in reserve as a cross-check. The hybrid is deliberate, not a hedge.

Why model-based is the right primary approach. The decision problem has three features that strongly favor a model:

- **Limited data, structured uncertainty.** Even with 36 years of monthly data, I have roughly 8-12 actual regime transitions in-sample. Model-free methods like Q-learning need many more interactions with each (state, action) pair to converge, and the regime transitions are precisely the rare events the policy needs to handle well. The HMM bakes in the structural assumption that regimes exist and persist, which is a strong prior that the data confirm but couldn't establish from scratch.
- **Interpretability.** A model-based policy can be queried: "What does the model think the current regime is, and why?" That matters both for diagnostic purposes (if the policy goes risk-off, I want to know whether VIX spiked, the term spread inverted, or credit spreads widened) and for any eventual stakeholder presentation. A model-free Q-function trained on the same observations gives a number with no obvious decomposition into regime belief and conditional optimal action.
- **Off-policy data is plentiful, on-policy data is impossible.** I cannot generate fresh trajectories of market regimes by acting. Every backtest re-uses the same historical observation sequence, just with a different policy on top. This is exactly the setting where model-based RL shines: fit the model once from observation, then plan against it under any policy.

Where model-free still helps. I am also running tabular Q-learning on a discretized state defined by recent observation statistics (rolling VIX percentile, rolling term spread, etc.) as a baseline. If the model-free policy substantially outperforms the model-based one out of sample, that is a signal of HMM misspecification, most likely from the 2-state assumption being too coarse. If it

underperforms, that is consistent with my hypothesis that the latent structure is doing real work.

Solution algorithm. QMDP with one-step lookahead, exactly as described in HW2. The known limitation is that QMDP underweights the value of active information-gathering (it assumes uncertainty resolves immediately after the next observation), but in this asset-allocation setting the agent isn't actively probing the regime; it's reacting to passively-arriving macro data. The information-gathering value is low and QMDP's bias is small in practice.